

Equivalencias de unidades y vínculos renglón–movimiento

Esquema de base de datos: en qué tablas se guarda cada cosa, y cómo se combinan en “Vincular por bloques”

1. Resumen

Tres piezas de datos intervienen en “Vincular por bloques”: el **Movimiento** (el kg cargado en báscula/planilla), el **ComprobanteRenglon** (la línea del comprobante fiscal que lo factura, con su **ComprobanteRenglonDetalle** de cantidad/unidad/precio), y el vínculo entre ambos (**ComprobanteRenglonMovimiento**). Una cuarta tabla, **ProductoEquivalenciaKg**, es la que resuelve cuando el movimiento y/o el renglón no están en Kg: guarda cuántos Kg equivale 1 unidad de otra unidad de medida, para un producto puntual.

Punto importante: la conversión a Kg NO se guarda en ningún lado de forma permanente sobre el movimiento o el renglón — se calcula al vuelo cada vez que la pantalla los necesita, leyendo **ProductoEquivalenciaKg**. Lo único que SÍ queda grabado en Kg de forma permanente es **ComprobanteRenglonMovimiento.cantidad_kg**, el monto que ESE vínculo puntual cubre — ver sección 4.

2. Tablas que ya existían (legacy, managed=False)

Estas tablas son anteriores a este sistema y las administra el código a mano (sin migraciones de Django) — cualquier cambio de estructura va por un script SQL corrido directamente en producción, no por makemigrations.

movimiento

Modelo Django: `movimientos.models.Movimiento`. Guarda cada movimiento de producto (Kg u otra unidad).

Columna	Tipo / referencia	Notas
<code>id_movimiento</code>	entero, PK	Autonumérico.
<code>id_producto</code>	FK → <code>producto_detalle</code>	
<code>total</code>	<code>decimal(12,2)</code>	Cantidad cargada, en la unidad de 'id_unidad_de_medida' — NO siempre Kg.
<code>id_unidad_de_medida</code>	FK → <code>comprobante_unidad_de_medida</code>	Puede ser Kg, Bolsa, Bolsón o cualquier otra — sin restricción de catálogo.
<code>id_entidad_emisor / id_entidad_receptor</code>	FK → <code>entidad</code>	Uno de los dos lados suele ser Fontana S.A.
<code>fecha, numero</code>	<code>fecha / entero</code>	Usados para ordenar (FIFO) y para el número de referencia.

comprobante_renglon + comprobante_renglon_detalle

Modelos Django: `comprobantes.models.ComprobanteRenglon` y `ComprobanteRenglonDetalle` (1 a 1, unidades por `id`). Cada renglón es una línea de un comprobante fiscal (factura, nota de crédito, etc.).

Columna	Tabla	Notas
id	comprobante_renglon (PK)	También PK de comprobante_renglon_detalle (OneToOne).
id_comprobante	comprobante_renglon	FK → comprobante.
id_producto	comprobante_renglon	FK → producto_detalle.
total	comprobante_renglon	Monto en \$ del renglón.
cantidad	comprobante_renglon_de_talle	Cantidad facturada, en la unidad de abajo — NO siempre Kg.
id_unidad_de_medida	comprobante_renglon_de_talle	FK → comprobante_unidad_de_medida. Puede ser Kg, “Unidad”, etc. — NUNCA Bolsa/Bolsón (ver más abajo).

comprobante_unidad_de_medida

Catálogo compartido de unidades — modelo `comprobantes.models.ComprobanteUnidadDeMedida` (id alfanumérico corto, ej. '01' = Kilogramos). La mayoría son unidades reales de AFIP, pero se agregaron a mano dos ids extra, 'BN' (Bolsón) y 'BS' (Bolsa) — constante `IDS_UNIDADES_SOLO_REMITOS` en `comprobantes/models.py`, migración `0003_seed_unidades_remitos.py`.

Nota: Bolsa y Bolsón sólo se usan en Movimiento/RemitoRenglon — el formulario de renglón de comprobante (`ComprobanteRenglonDetalleForm`) las excluye a propósito de su desplegable, porque un comprobante fiscal real nunca puede facturar en esas unidades. Cualquier OTRA unidad del catálogo (Kg, Unidad, etc.) sí puede aparecer de los dos lados.

3. Tabla nueva: equivalencias (managed=True)

cta_cte_producto_equivalencia_kg

Modelo Django: `cuenta_corriente_productos.models.ProductoEquivalenciaKg`. Una fila = “para este producto, 1 unidad de ESTA unidad equivale a tantos Kg”. Se carga y edita desde la pantalla “Equivalencias de unidades”.

Columna	Tipo / referencia	Notas
id	entero, PK	Autonumérico (tabla nueva, Django la administra entera).
producto_id	FK → producto_detalle	on_delete=CASCADE.
unidad_id	FK → comprobante_unidad_de_medida	on_delete=CASCADE. Nunca puede ser Kg (excluida en el formulario).
factor_kg	decimal(12,4)	Cuántos Kg equivale 1 unidad de 'unidad_id' para este producto.
guardado_el / modificado_el	datetime	auto_now_add / auto_now.

Restricción única: `UniqueConstraint(producto, unidad)` — no puede haber dos filas para la misma combinación producto+unidad (la pantalla de alta lo bloquea con un mensaje). Un mismo producto SÍ puede tener varias filas, una por cada unidad distinta que use — por ejemplo, producto 1088 tiene hoy una fila para Bolsa (factor 8) y otra para “Unidad” (factor 9, proveedor Purralef).

Nota: Es una tabla `managed=True`: cuando se crea o cambia de estructura, Gastón corre `makemigrations/migrate` él mismo (convención del proyecto) — a diferencia de las tablas *legacy* de la sección 2, que se modifican con SQL a mano.

4. Tabla nueva: el vínculo renglón–movimiento

`cta_cte_comprobante_renglon_movimiento`

Modelo Django: `cuenta_corriente_productos.models.ComprobanteRenglonMovimiento`. Cada fila conecta UN renglón de comprobante con UN movimiento, y registra cuánto de ese movimiento cubre ESE vínculo puntual.

Columna	Tipo / referencia	Notas
<code>id</code>	entero, PK	
<code>renglon_id</code>	FK → <code>comprobante_renglon</code>	<code>on_delete=CASCADE</code> .
<code>movimiento_id</code>	FK → <code>movimiento</code>	<code>on_delete=CASCADE</code> .
<code>cantidad_kg</code>	<code>decimal(14,2)</code> , puede ser NULL	Cuánto de ESTE vínculo, SIEMPRE en Kg (ya convertido) — ver más abajo.
<code>observaciones</code>	texto corto	Opcional.
<code>guardado_el</code>	<code>datetime</code>	<code>auto_now_add</code> .

Restricción única: `UniqueConstraint(renglon, movimiento)` — no se puede vincular dos veces el mismo par renglón+movimiento (si hace falta corregir el monto, hay que desvincular y volver a vincular). Un mismo movimiento puede tener VARIOS renglones vinculados a lo largo del tiempo (ej. anticipo + ajustes de precio), y un mismo renglón puede cubrir a su vez varios movimientos, si su capacidad alcanza.

Nota: `cantidad_kg` en NULL significa “vínculo viejo, de antes de que este campo existiera” — se interpreta como “cubre el movimiento completo” (compatibilidad hacia atrás). Todo vínculo creado desde las pantallas actuales (Vincular por bloques / Vincular renglón) siempre completa un valor real.

5. Cómo se combinan: la conversión a Kg en tiempo real

Ni `movimiento.total` ni `comprobante_renglon_detalle.cantidad` se tocan nunca para convertirlos — quedan tal cual se cargaron, en su unidad original. La conversión ocurre sólo en el código Python, cada vez que la pantalla arma su listado, con dos funciones espejadas (ambas en `cuenta_corriente_productos/views.py`):

Función	Convierte	Cómo
<code>_total_movimiento_en_kg(movimiento)</code>	<code>movimiento.total</code>	Si <code>unidad_de_medida_id</code> ya es Kg ('01'), lo devuelve tal cual. Si no, busca en <code>cta_cte_producto_equivalencia_kg</code> la fila (<code>producto_id</code> , <code>unidad_id</code>) del movimiento; si existe, devuelve <code>total × factor_kg</code> . Si no existe ninguna fila para esa combinación, lo devuelve SIN convertir (comportamiento de siempre).
<code>_total_renglon_en_kg(renglon)</code>	<code>comprobante_renglon_detalle.cantidad</code>	Misma lógica, mirando (<code>producto_id</code> , <code>unidad_id</code>) del renglón.

Estas dos funciones alimentan todo lo demás: el filtro de “Vincular por bloques” (qué movimientos/renglones aparecen), el cálculo de “pendiente” y “disponible”, la columna “Diferencia”, y “Pendientes por producto”. Ambos lados (movimiento y renglón) se resuelven de forma independiente — pueden estar los dos en Kg, uno solo convertido, o los dos convertidos con equivalencias distintas (por ejemplo, movimiento en Bolsa ×8 contra renglón en “Unidad” ×9).

Al confirmar “Vincular”, el monto que efectivamente se graba en `ComprobanteRenglonMovimiento.cantidad_kg` es siempre el resultado YA CONVERTIDO a Kg de estas funciones (o la parte de él que entra en la capacidad libre del renglón) — nunca la cifra cruda de `movimiento.total` en su unidad original.

Ejemplo real: producto 1088, dos proveedores

Datos de las cuatro tablas para el caso que motivó este documento (valores simplificados):

Tabla	Fila
<code>cta_cte_producto_equivalencia_kg</code>	<code>producto_id=1088</code> , <code>unidad_id='BS'</code> (Bolsa), <code>factor_kg=8</code>
<code>cta_cte_producto_equivalencia_kg</code>	<code>producto_id=1088</code> , <code>unidad_id='UN'</code> (Unidad), <code>factor_kg=9</code>
<code>movimiento</code> (Don Basilio)	<code>id_producto=1088</code> , <code>id_unidad_de_medida='BS'</code> , <code>total=1</code> → <code>_total_movimiento_en_kg = 1 × 8 = 8 Kg</code>
<code>comprobante_renglon_detalle</code> (Don Basilio)	<code>cantidad=20</code> , <code>id_unidad_de_medida='01'</code> (Kg) → ya está en Kg, no se convierte
<code>movimiento</code> (Purralef)	<code>id_producto=1088</code> , <code>id_unidad_de_medida='01'</code> (Kg), <code>total=18</code> → ya está en Kg
<code>comprobante_renglon_detalle</code> (Purralef)	<code>cantidad=2</code> , <code>id_unidad_de_medida='UN'</code> → <code>_total_renglon_en_kg = 2 × 9 = 18 Kg</code>
<code>cta_cte_comprobante_renglon_movimiento</code>	al vincular cada par: <code>cantidad_kg</code> queda en 8 (caso Don Basilio) y en 18 (caso Purralef) — siempre en Kg, sin importar en qué unidad estaba cargado cada lado

6. Resumen de tablas

Modelo Django	Tabla real	managed
Movimiento	movimiento	False (legacy)
ComprobanteRenglon	comprobante_renglon	False (legacy)
ComprobanteRenglonDetalle	comprobante_renglon_detalle	False (legacy)
ComprobanteUnidadDeMedida	comprobante_unidad_de_medida	False (legacy, con 2 ids agregados a mano: BN, BS)
ProductoEquivalenciaKg	cta_cte_producto_equivalencia_kg	True (nueva)
ComprobanteRenglonMovimiento	cta_cte_comprobante_renglon_movimiento	True (nueva)

Las dos tablas managed=True (equivalencias y vínculos) son enteramente del dominio de este sistema — cualquier cambio de estructura futuro pasa por makemigrations/migrate. Las legacy se modifican, cuando hace falta, con un script SQL a mano corrido directamente en producción.